PROJECT-STRUCTURE.md — PrepGenie

1. Complete Folder Structure

```
prepgenie/
├── client/                          # React frontend (Vite)
│   ├── public/
│   │   └── favicon.svg
│   ├── src/
│   │   ├── assets/                  # images, icons, illustrations
│   │   ├── components/              # reusable, presentational components
│   │   │   ├── ui/                  # shadcn/ui primitives (Button, Card,
Accordion...)
│   │   │   ├── Navbar.jsx
│   │   │   ├── Footer.jsx
│   │   │   ├── ProtectedRoute.jsx
│   │   │   ├── ErrorBoundary.jsx
│   │   │   ├── SkeletonLoader.jsx
│   │   │   ├── UploadDropzone.jsx
│   │   │   ├── AnalysisReport.jsx
│   │   │   ├── ScoreGauge.jsx
│   │   │   ├── ChatBubble.jsx
│   │   │   ├── ResumeHistoryChart.jsx
│   │   │   ├── SkillRadarChart.jsx
│   │   │   └── StreakTracker.jsx
│   │   ├── pages/                   # route-level components
│   │   │   ├── Landing.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Signup.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Analyzer.jsx
│   │   │   ├── RoleSelect.jsx
│   │   │   ├── InterviewChat.jsx
│   │   │   ├── InterviewResults.jsx
│   │   │   ├── Settings.jsx
│   │   │   └── NotFound.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx      # global auth state via Supabase session
│   │   ├── hooks/
│   │   │   ├── useSpeechToText.js
│   │   │   └── useApi.js            # wraps apiClient with loading/error
state
│   │   ├── lib/
│   │   │   ├── supabaseClient.js
│   │   │   └── apiClient.js         # axios instance, attaches JWT, handles
errors
```

```
|   |   ├── App.jsx                  # router setup
|   |   ├── main.jsx
|   |   └── index.css                # Tailwind directives + design tokens
|   ├── .env.example
|   ├── index.html
|   ├── tailwind.config.js
|   ├── vite.config.js
|   └── package.json
|
├── server/                          # Express backend
|   ├── routes/
|   |   ├── authRoutes.js
|   |   ├── resumeRoutes.js
|   |   ├── interviewRoutes.js
|   |   └── dashboardRoutes.js
|   ├── controllers/                 # request handling logic, thin —
delegates to services
|   |   ├── resumeController.js
|   |   ├── interviewController.js
|   |   └── dashboardController.js
|   ├── services/                    # core business logic, reusable,
testable
|   |   ├── aiService.js             # all Claude API calls + prompt
templates
|   |   ├── interviewService.js      # question generation + scoring logic
|   |   └── parseResume.js           # pdf-parse/mammoth extraction logic
|   ├── middleware/
|   |   ├── authMiddleware.js        # verifies Supabase JWT
|   |   ├── errorHandler.js          # centralized error → JSON envelope
|   |   └── rateLimiter.js           # express-rate-limit config for AI
routes
|   ├── utils/
|   |   ├── safeParseAIJson.js       # strips markdown fences, validates JSON
shape
|   |   └── validators.js            # request body validation helpers
|   ├── config/
|   |   └── supabaseAdmin.js         # server-side Supabase client (service
role key)
|   ├── .env.example
|   ├── index.js                     # Express app entrypoint
|   └── package.json
|
├── docs/                             # architecture & planning artifacts
(this deliverable set)
|   ├── 01-Executive-Summary.md
|   ├── 02-PRD.md
|   ├── 03-Implementation-Blueprint-Day2-10.md
```

```
|   ├── 04-Pitch-Deck.md
|   ├── ARCHITECTURE.md
|   ├── SCHEMA.md
|   ├── API.md
|   ├── UI-WIREFRAMES.md
|   ├── PROJECT-STRUCTURE.md
|   ├── architecture-diagram.png
|   └── demo.gif
|
├── .gitignore
├── LICENSE
└── README.md
```

---

2. Explanation of Every Major Folder

`client/src/components/`

Purely presentational, reusable UI pieces with no route-level logic. Split further into `ui/` (shadcn primitives) so design-system pieces are visually distinct from feature-specific components (e.g. `AnalysisReport.jsx`).

`client/src/pages/`

One component per route. Pages compose components together and own page-level data fetching (via `useApi` hook). Keeping pages "dumb composers" and components "reusable" is a standard React separation-of-concerns pattern that's easy to explain in an interview.

`client/src/context/`

Global state that must be available app-wide (auth session) lives here via React Context, avoiding prop-drilling without introducing a heavier state library (Redux/Zustand) that would be overkill for this app's size.

`client/src/hooks/`

Custom hooks encapsulate reusable stateful logic (`useApi` standardizes loading/error/data state for every API call; `useSpeechToText` isolates the Web Speech API browser quirks) — keeps components clean.

`client/src/lib/`

Singleton client instances (`supabaseClient.js`, `apiClient.js`) configured once and imported everywhere, avoiding re-instantiation and keeping config (base URLs, interceptors) centralized.

`server/routes/`

Thin route definition files — map HTTP verb + path to a controller function. No business logic here, purely wiring.

`server/controllers/`

Handle request/response concerns (parsing `req.body`, calling services, shaping the response envelope, calling `next(err)` on failure). Controllers stay thin and delegate real work to `services/`.

`server/services/`

Where the actual business logic and external API calls live (AI prompting, PDF parsing, scoring logic). This is the layer that would be unit-tested first if tests were added, and it's fully decoupled from Express (`req`/`res`) — making it reusable (e.g., in a future CLI tool or background job).

`server/middleware/`

Cross-cutting concerns applied across many routes: auth verification, centralized error handling, and rate limiting — kept separate from business logic per Express best practice.

`server/utils/`

Small, pure, stateless helper functions (JSON-safety parsing, input validators) — no side effects, easy to reason about independently.

`server/config/`

Environment-dependent client setup (Supabase admin client using the service-role key, which must **never** be used client-side) — isolated here so it's obvious this file handles sensitive credentials.

`docs/`

All planning artifacts (PRD, architecture, schema, API spec, wireframes) live in the repo itself — this is a deliberate choice that signals engineering maturity to anyone (recruiter or teammate) browsing the GitHub repo, and gives Day 2–10 implementation a single source of truth to build against.

---

3. Why This Architecture Was Chosen

1. **Clear separation of concerns (routes → controllers → services)** mirrors patterns used in real production Node.js codebases, making this project a legitimate talking point for backend-focused interview questions ("how did you structure your API layer?").
2. **Feature-based frontend organization (pages + components + hooks)** is the standard React convention most hiring teams expect, minimizing onboarding friction if this code were ever handed to a teammate.
3. **All secrets isolated to `server/config/` and `.env`** files (never committed, `.env.example` provided instead) demonstrates secure-by-default thinking — a strong signal in any technical interview.
4. **`docs/` folder committed to the repo** turns the entire planning process (this document set) into visible engineering artifacts on GitHub — recruiters and

interviewers can see the PRD → Architecture → Implementation chain, not just the final code.

5. **No premature abstraction** — no unnecessary state management libraries, no microservices, no over-engineered folder nesting. The structure is exactly as complex as a 10-day, single-developer AI SaaS project needs to be, and no more — directly reflecting the "avoid scope creep" principle from the Executive Summary.